

- Project 1: E-Commerce Platform on AWS EKS — PoC Architecture
 - 1. Business Context
 - 2. High-Level Architecture
 - 3. Detailed Request Flow (Step-by-Step)
 - Example: Customer creates an order — POST /api/v1/orders
 - 4. VARIANT A: NLB-Based Architecture
 - When to Use NLB
 - Architecture Diagram (NLB)
 - Kubernetes Manifests (NLB Variant)
 - 5. VARIANT B: ALB-Based Architecture
 - When to Use ALB
 - Architecture Diagram (ALB)
 - Kubernetes Manifests (ALB Variant)
 - 6. NLB vs ALB — Comparison
 - 7. Service Mesh (Istio) Configuration
 - Istio DestinationRule (Circuit Breaker + Connection Pool)
 - Istio PeerAuthentication (Enforce mTLS)
 - Istio AuthorizationPolicy (Service-to-Service ACL)
 - 8. Kubernetes Service & Deployment (Shared Across Both Variants)
 - 9. Aurora DB Architecture
 - 10. Security Layers Summary

Project 1: E-Commerce Platform on AWS EKS — PoC Architecture

Flow: Route53 → API Gateway (WAF) → PrivateLink → EKS → Aurora DB

Routing: Path-based REST API

Variants: NLB-based vs ALB-based (separate sections)

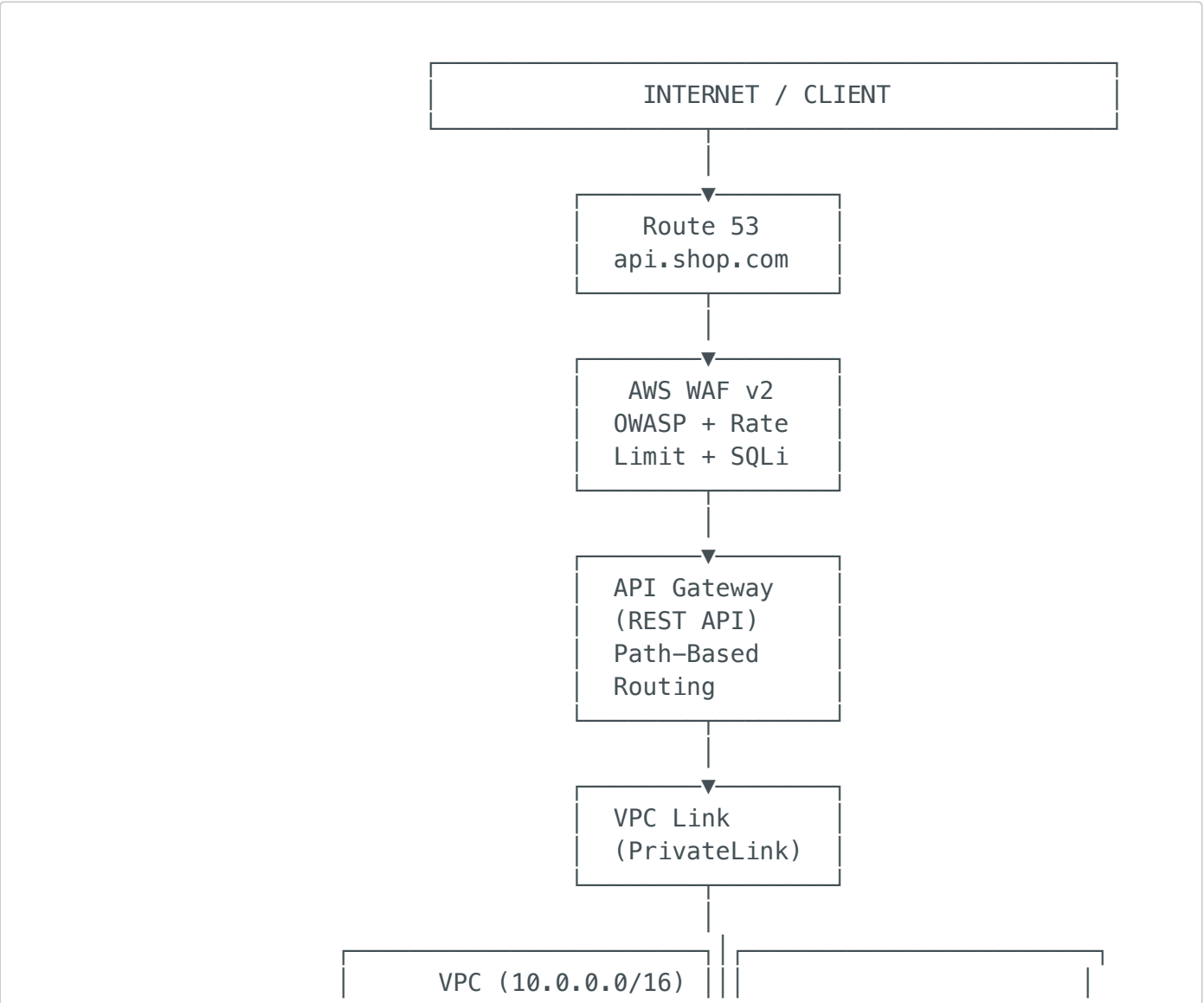
Includes: Ingress, Services, Service Mesh (Istio)

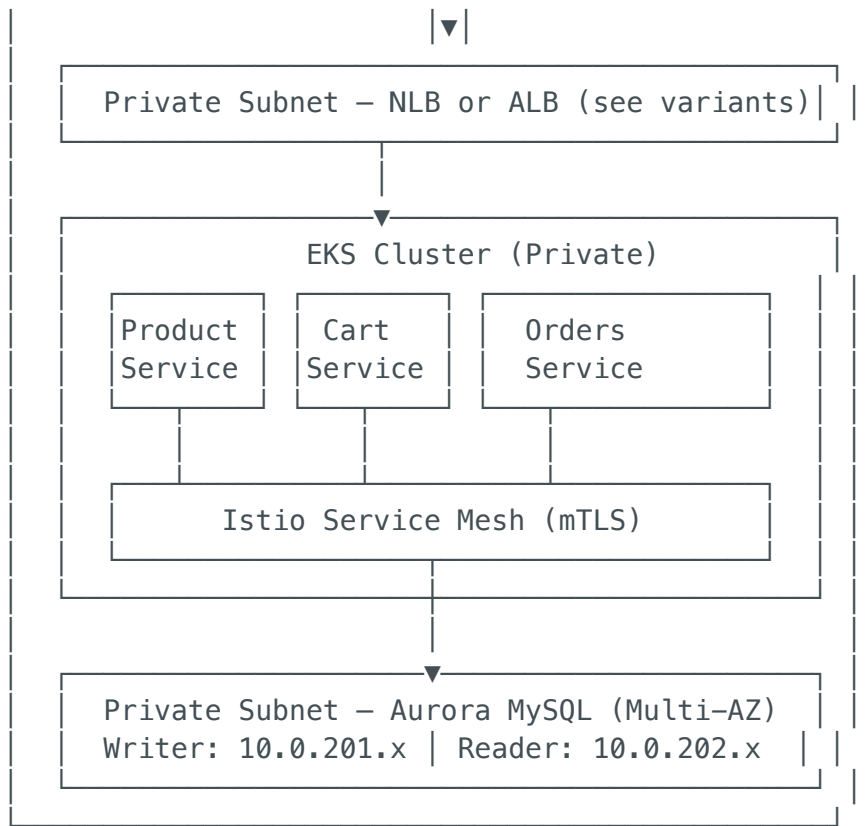
1. Business Context

An e-commerce platform with the following REST API services:

Service	Path	Method	Description
Product Catalog	<code>/api/v1/products</code>	GET, GET/:id	Browse products
Cart	<code>/api/v1/cart</code>	GET, POST, PUT, DELETE	Manage shopping cart
Orders	<code>/api/v1/orders</code>	POST, GET/:id	Create and track orders
Payments	<code>/api/v1/payments</code>	POST	Process payments
Users	<code>/api/v1/users</code>	POST, GET, PUT	User registration & profile
Inventory	<code>/api/v1/inventory</code>	GET/:sku, PUT	Stock management

2. High-Level Architecture





3. Detailed Request Flow (Step-by-Step)

Example: Customer creates an order — **POST /api/v1/orders**

Step 1: Client sends HTTPS request to `api.shop.com`

Step 2: Route53 resolves `api.shop.com` → API Gateway endpoint

Step 3: AWS WAF inspects request

- Rule 1: Rate limit check (2000 req/5min per IP) → PASS
- Rule 2: OWASP Common Rules (XSS, LFI) → PASS
- Rule 3: SQLi detection → PASS
- Rule 4: Known bad inputs (Log4j) → PASS

Step 4: API Gateway receives request

- Matches route: `POST /api/v1/orders`
- Authenticates: Cognito JWT or Lambda Authorizer validates token
- Request validation: JSON schema check on request body
- Routes to VPC Link integration

Step 5: VPC Link (PrivateLink) forwards to NLB/ALB in private subnet
(No internet traversal – stays within AWS backbone)

Step 6: NLB/ALB routes to EKS Service (see Variant A / Variant B below)

Step 7: Istio sidecar (envoy proxy) intercepts request

- └ mTLS: Verifies caller identity
- └ Rate limiting: Per-service rate limit
- └ Retry policy: 3 retries on 5xx with exponential backoff
- └ Circuit breaker: Opens if >50% error rate

Step 8: Order Service pod processes request

- └ Validates order payload
- └ Calls Inventory Service (via mesh): GET /api/v1/inventory/{sku}
 - └ Istio routes to inventory-

service.ecommerce.svc.cluster.local

- └ Inventory pod checks Aurora Reader → stock available
- └ Calls Payment Service (via mesh): POST /api/v1/payments
 - └ Payment pod calls Stripe API via NAT Gateway
 - └ Returns payment_token
- └ Writes order record to Aurora Writer endpoint
 - └ Connection via IRSA (no hardcoded credentials)
 - └ Aurora Writer: ecommerce-db.cluster-xxx.us-east-

1.rds.amazonaws.com

Step 9: Order Service returns 201 Created with order_id

Step 10: Response flows back: Pod → Istio → ALB/NLB → VPC Link
→ API Gateway → Client

Step 11: Async post-processing:

- └ Order Service publishes event to EventBridge: "order.created"
- └ EventBridge → SQS → Lambda → Send confirmation email (SES)
- └ EventBridge → SQS → Lambda → Update ERP system

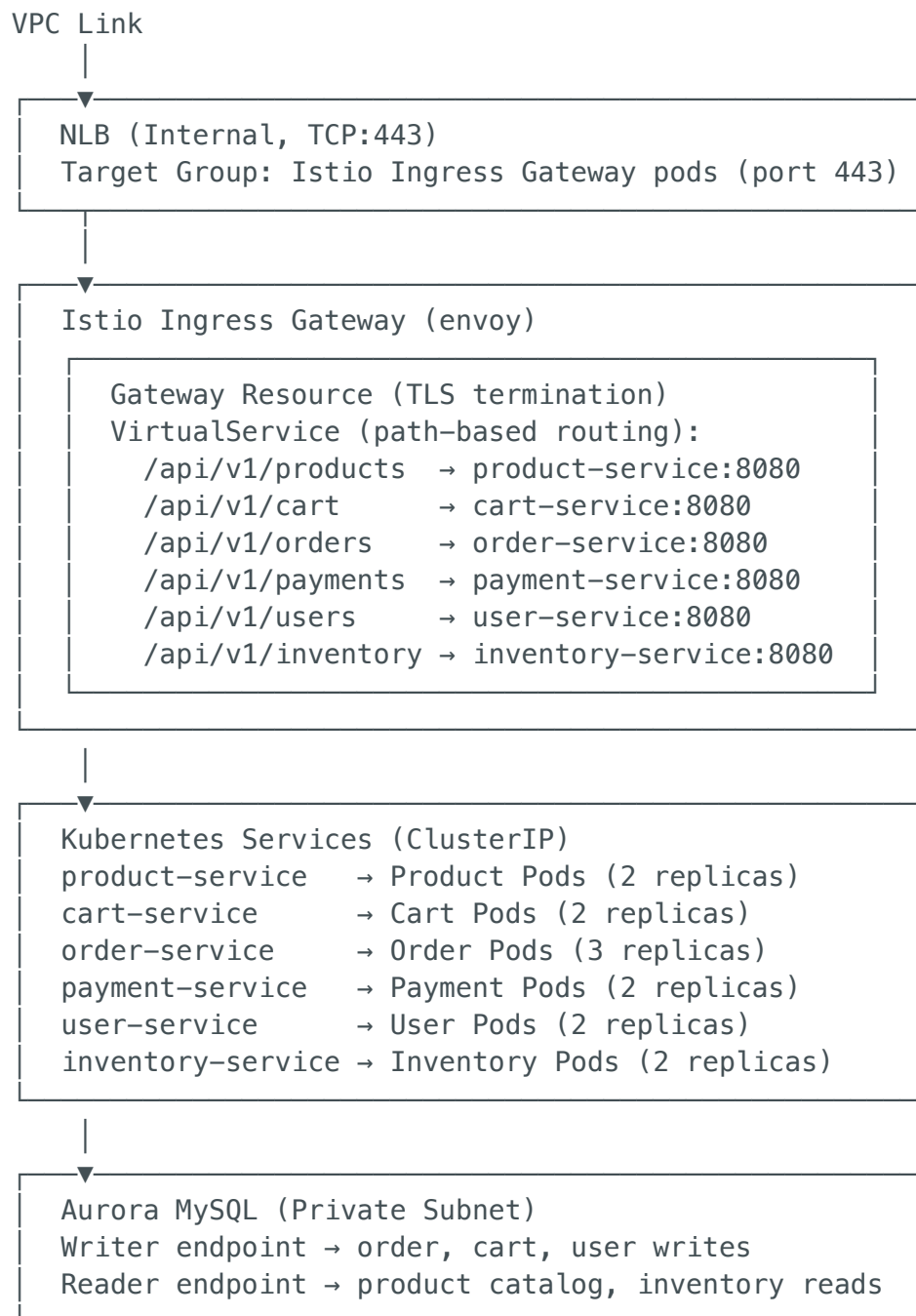
4. VARIANT A: NLB-Based Architecture

When to Use NLB

- API Gateway VPC Link **requires NLB** for REST API type (not HTTP API)
- Layer 4 (TCP) load balancing — no HTTP-aware routing at LB level
- Ultra-low latency (<100µs at LB layer)
- Path-based routing happens **inside EKS** (Istio VirtualService)

Architecture Diagram (NLB)

API Gateway (REST)



Kubernetes Manifests (NLB Variant)

Istio Gateway + VirtualService:

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: ecommerce-gateway
  namespace: ecommerce
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 443
```

```
    name: https
    protocol: HTTPS
  tls:
    mode: SIMPLE
    credentialName: ecommerce-tls-cert
  hosts:
    - "api.shop.com"
```

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: ecommerce-routing
  namespace: ecommerce
spec:
  hosts:
    - "api.shop.com"
  gateways:
    - ecommerce-gateway
  http:
    - match:
        - uri:
            prefix: /api/v1/products
      route:
        - destination:
            host: product-service
            port:
              number: 8080
    - match:
        - uri:
            prefix: /api/v1/cart
      route:
        - destination:
            host: cart-service
            port:
              number: 8080
    - match:
        - uri:
            prefix: /api/v1/orders
      route:
        - destination:
            host: order-service
            port:
              number: 8080
          weight: 90
        - destination:
            host: order-service-canary
            port:
              number: 8080
          weight: 10 # 10% canary traffic
    - match:
        - uri:
            prefix: /api/v1/payments
      route:
        - destination:
            host: payment-service
            port:
              number: 8080
```

```

    retries:
      attempts: 3
      perTryTimeout: 5s
  - match:
      - uri:
          prefix: /api/v1/users
    route:
      - destination:
          host: user-service
          port:
            number: 8080
  - match:
      - uri:
          prefix: /api/v1/inventory
    route:
      - destination:
          host: inventory-service
          port:
            number: 8080

```

NLB Service for Istio Ingress Gateway:

```

apiVersion: v1
kind: Service
metadata:
  name: istio-ingressgateway
  namespace: istio-system
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"
    service.beta.kubernetes.io/aws-load-balancer-internal: "true"
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internal"
    service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-balancing-
enabled: "true"
spec:
  type: LoadBalancer
  selector:
    istio: ingressgateway
  ports:
    - name: https
      port: 443
      targetPort: 8443
      protocol: TCP

```

5. VARIANT B: ALB-Based Architecture

When to Use ALB

- API Gateway **HTTP API** type supports ALB via VPC Link natively
- Layer 7 (HTTP) routing — ALB handles path-based routing before EKS
- AWS ALB Ingress Controller manages ALB lifecycle from K8s
- Native WAF integration at ALB layer (additional to API GW WAF)

Architecture Diagram (ALB)



Kubernetes Manifests (ALB Variant)

ALB Ingress (AWS Load Balancer Controller):

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce

```



```
annotations:
  kubernetes.io/ingress.class: alb
  alb.ingress.kubernetes.io/scheme: internal
  alb.ingress.kubernetes.io/target-type: ip
  alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS": 443}]'
  alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-
1:123456:certificate/xxx
  alb.ingress.kubernetes.io/healthcheck-path: /actuator/health
  alb.ingress.kubernetes.io/healthcheck-interval-seconds: "15"
  alb.ingress.kubernetes.io/group.name: ecommerce
spec:
  rules:
    - host: api.shop.com
      http:
        paths:
          - path: /api/v1/products
            pathType: Prefix
            backend:
              service:
                name: product-service
                port:
                  number: 8080
          - path: /api/v1/cart
            pathType: Prefix
            backend:
              service:
                name: cart-service
                port:
                  number: 8080
          - path: /api/v1/orders
            pathType: Prefix
            backend:
              service:
                name: order-service
                port:
                  number: 8080
          - path: /api/v1/payments
            pathType: Prefix
            backend:
              service:
                name: payment-service
                port:
                  number: 8080
          - path: /api/v1/users
            pathType: Prefix
            backend:
              service:
                name: user-service
                port:
                  number: 8080
          - path: /api/v1/inventory
            pathType: Prefix
            backend:
              service:
                name: inventory-service
                port:
                  number: 8080
```

6. NLB vs ALB — Comparison

Feature	NLB (Variant A)	ALB (Variant B)
OSI Layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Path routing	Done by Istio VirtualService inside EKS	Done by ALB rules before reaching EKS
API GW VPC Link	Required for REST API type	Works with HTTP API type
Latency	~100µs (ultra-low)	~1-5ms (HTTP processing)
TLS termination	At Istio Ingress Gateway	At ALB
WAF support	Not natively (WAF at API GW only)	Yes (WAF on ALB too)
Canary routing	Istio VirtualService weight-based	Weighted target groups
Health checks	TCP-level	HTTP-level (smarter)
Cost	Lower (\$0.006/NLCU-hr)	Higher (\$0.008/LCU-hr)
Service mesh role	Full routing + mTLS + observability	mTLS + observability only (routing at ALB)
When to use	High-throughput, low-latency, full mesh control	Simpler setup, AWS-native routing, WAF at LB

7. Service Mesh (Istio) Configuration

Istio DestinationRule (Circuit Breaker + Connection Pool)

```

apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: order-service-dr
  namespace: ecommerce
spec:
  host: order-service
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        h2UpgradePolicy: DEFAULT
        http1MaxPendingRequests: 50
        http2MaxRequests: 100
        maxRetries: 3
    outlierDetection:
      consecutive5xxErrors: 5      # Circuit breaker
                                   # Trip after 5 consecutive 5xx
      interval: 10s                # Check every 10s
      baseEjectionTime: 30s        # Eject unhealthy pod for 30s
      maxEjectionPercent: 50       # Max 50% pods ejected
    tls:
      mode: ISTIO_MUTUAL           # Automatic mTLS

```

Istio PeerAuthentication (Enforce mTLS)

```

apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ecommerce
spec:
  mtls:
    mode: STRICT    # All traffic must be mTLS

```

Istio AuthorizationPolicy (Service-to-Service ACL)

```

apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: payment-service-policy
  namespace: ecommerce
spec:
  selector:

```

```

    matchLabels:
      app: payment-service
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/ecommerce/sa/order-service"]
      to:
        - operation:
            methods: ["POST"]
            paths: ["/api/v1/payments"]
# Only order-service can call payment-service. All other traffic is denied.

```

8. Kubernetes Service & Deployment (Shared Across Both Variants)

```

# --- Namespace ---
apiVersion: v1
kind: Namespace
metadata:
  name: ecommerce
  labels:
    istio-injection: enabled # Auto-inject Istio sidecar
---
# --- Order Service Deployment ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
  namespace: ecommerce
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
        version: v1
    spec:
      serviceAccountName: order-service-sa # IRSA for Aurora access
      securityContext:
        runAsNonRoot: true
        fsGroup: 1000
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:

```

```

    matchLabels:
      app: order-service
containers:
  - name: order-service
    image: 123456.dkr.ecr.us-east-1.amazonaws.com/order-service:abc123
    ports:
      - containerPort: 8080
    env:
      - name: AURORA_WRITER_ENDPOINT
        valueFrom:
          secretKeyRef:
            name: aurora-credentials
            key: writer-endpoint
      - name: AURORA_READER_ENDPOINT
        valueFrom:
          secretKeyRef:
            name: aurora-credentials
            key: reader-endpoint
    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
        port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
        initialDelaySeconds: 60
        periodSeconds: 15
    resources:
      requests:
        cpu: 250m
        memory: 512Mi
      limits:
        cpu: "1"
        memory: 1Gi
    securityContext:
      readOnlyRootFilesystem: true
      allowPrivilegeEscalation: false

```

--- ClusterIP Service ---

```

apiVersion: v1
kind: Service
metadata:
  name: order-service
  namespace: ecommerce
spec:
  type: ClusterIP
  selector:
    app: order-service
  ports:
    - name: http
      port: 8080
      targetPort: 8080

```

9. Aurora DB Architecture

Aurora MySQL Cluster (Multi-AZ)

Writer Instance (db.r6g.xlarge)
Endpoint: ecommerce.cluster-xxx.us-east-1.rds.amazonaws.com |
AZ: us-east-1a
Used by: order-service, cart-service, user-service

Reader Instance (db.r6g.large)
Endpoint: ecommerce.cluster-ro-xxx.us-east-1.rds.amazonaws.com |
AZ: us-east-1b
Used by: product-service, inventory-service (reads)

Security:

- Private subnet only (no public access)
- Security group: allow 3306 from EKS node SG only
- KMS CMK encryption at rest
- IAM DB authentication via IRSA (no passwords)
- Automated backups (35-day retention)
- Performance Insights enabled

10. Security Layers Summary

Layer	Component	Protection
1	Route53 + ACM	HTTPS/TLS 1.3, DNSSEC
2	WAF v2	OWASP Top 10, SQLi, rate limiting
3	API Gateway	JWT auth, request validation, throttling
4	PrivateLink	No internet traversal to backend
5	NLB/ALB	Internal only, health checks
6	Istio mTLS	Encrypted service-to-service
7	Istio AuthzPolicy	Service-level ACLs
8	NetworkPolicy	Pod-level network isolation
9	Pod Security	Non-root, read-only FS, no privilege escalation

Layer	Component	Protection
10	IRSA	Pod-level IAM (no shared node credentials)
11	Aurora	KMS encryption, IAM auth, private subnet
12	VPC Endpoints	ECR, S3, STS — no internet for AWS API calls